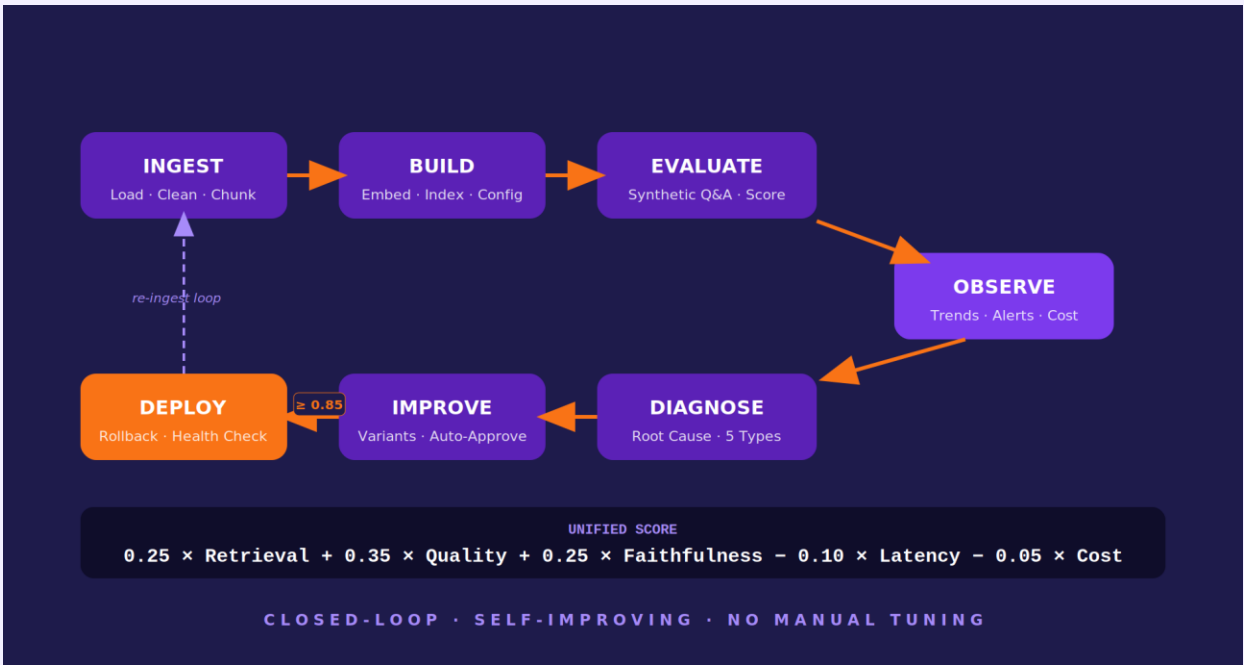


AutoRAG

Self-Improving RAG Platform with Autonomous Knowledge Compression

ENTERPRISE ARCHITECTURE DESIGN DOCUMENT

Version 1.0 · 15 May 2026 · Derived from PRD v2.3



Version 1.0 | 15 May 2026 | CONFIDENTIAL

Table of Contents

01. Executive Summary3

02. Architecture Goals & Principles4

03. High-Level Platform Architecture5

04. Core Component Architecture6

05. 7-Agent Autonomous Architecture8

06. Data & Vector Architecture10

07. AI / LLM Architecture12

08. Workflow & Sequence Flows13

09. Evaluation & Improvement Loop14

10. Security & Governance Architecture15

11. Deployment & Scalability Architecture16

12. Observability & LLMOps17

13. Resilience & Failover Design18

14. Technology Stack19

15. Architecture Decision Records20

16. Future Architecture Roadmap21

17. Conclusion22

01 Executive Summary

AutoRAG establishes a new infrastructure category — Autonomous AI Quality Systems — by implementing a 7-agent closed loop that builds, evaluates, observes, diagnoses, improves, and deploys RAG pipelines without manual tuning or labeled data.

This document describes the complete technical architecture of AutoRAG v1.0. It is written for engineers who will build and operate the system, but deliberately uses plain language so that beginners can follow the design reasoning. Every architectural decision links back to a specific requirement in PRD v2.3.

Think of AutoRAG like a factory quality control system. A car factory doesn't have a human manually inspect every car — it has automated sensors, diagnostic tools, and improvement processes. AutoRAG does the same for AI pipelines: automated measurement (Evaluator), anomaly detection (Observer), root-cause analysis (Diagnoser), and automated improvement (Improver).

1.1 What This Document Covers

Table 1: Architecture Document Scope

Architecture Domain	What It Explains	Key Diagrams
Platform Architecture	How the 5 layers work together — what each layer does and why	Figure 1: 5-Layer Stack
Agent Architecture	How the 7 agents communicate and what each one is responsible for	Figure 2: Agent Workflow
Data Architecture	Where data lives, how it flows, how embeddings are stored	Figure 3: Retrieval Flow
AI/LLM Architecture	Which AI providers are used, how failover works	Figure 4: LLM Fallback
Evaluation Loop	How quality is measured and how improvement is triggered	Figure 5: Eval Loop
Security	Authentication, encryption, audit logging, trust boundaries	Figure 6: Trust Boundary
Deployment	Development, staging, production, on-premise environments	Figure 7: Deploy Topology
Observability	Monitoring, alerting, LLMOps loop	Figure 8: Monitoring Arch
Resilience	Retry logic, circuit breakers, provider failover	Embedded in §13
ADRs	Why each technology was chosen over alternatives	Tables in §15

02 Architecture Goals & Principles

Every architecture decision in AutoRAG traces back to one of six guiding principles. These are not abstract values — each maps to a concrete engineering constraint.

Table 2: Architecture Principles and Engineering Constraints

Principle	Plain English	Engineering Constraint
Measure First	You cannot improve what you cannot measure. Quality is quantified before anything else.	Evaluator Agent runs before Diagnoser or Improver. Score stored in PostgreSQL before next step.
Stateless Agents	Each agent is an independent, replaceable unit. One agent failing does not break others.	All agent state persisted to PostgreSQL or MinIO before hand-off. No in-memory cross-agent state.
Zero Cloud Lock-in	AutoRAG runs anywhere — on a laptop, a K8s cluster, or air-gapped server.	Docker Compose for dev. MinIO (not S3). Qdrant (not Pinecone). Local LLM via Ollama as fallback.
Immutable Audit Trail	Every decision the system makes is permanently recorded and cannot be changed.	audit_log table is INSERT-ONLY. No UPDATE or DELETE ever permitted. 11 event types logged.
Safe by Default	The system defaults to asking for human approval rather than acting autonomously on risky changes.	Faithfulness < 0.5 blocks deploy. Model swaps always require Mandatory Gate. PII halts ingestion.
Progressive Disclosure	A beginner can use it in 60 seconds. An expert can configure everything.	5-question Configurator for beginners. Full YAML editable for experts. Same underlying engine.

2.1 Quality Targets from PRD v2.3

Table 3: Architecturally-Significant Performance Targets

SLO	Target	Architectural Implication
/query p95 latency	< 3 seconds	Qdrant HNSW search < 50ms + LLM < 2.5s. Embedding cached. Local fallback for latency spikes.
Evaluation cycle (50 Q&A)	< 5 minutes	Parallelised judge calls. Celery concurrency 2 per worker. LLM batching where supported.
Pipeline build time	< 5 minutes	Batch embeddings 100 chunks/call. Retry with backoff. MiniLM fallback if OpenAI rate-limited.
Configurator response	< 60 seconds	Rule-based engine (no LLM). Pure Python dict lookup. No external calls.
Unified Score accuracy	≥ 90% classification	Validated against 200-failure test set. 3× averaging on borderline scores (0.40–0.60 range).

03 High-Level Platform Architecture

AutoRAG is organised as five horizontal layers. Think of these as the floors of a building — each floor has a clear job, and they communicate only through defined interfaces. This makes it easy to scale, debug, and replace any layer without affecting the others.

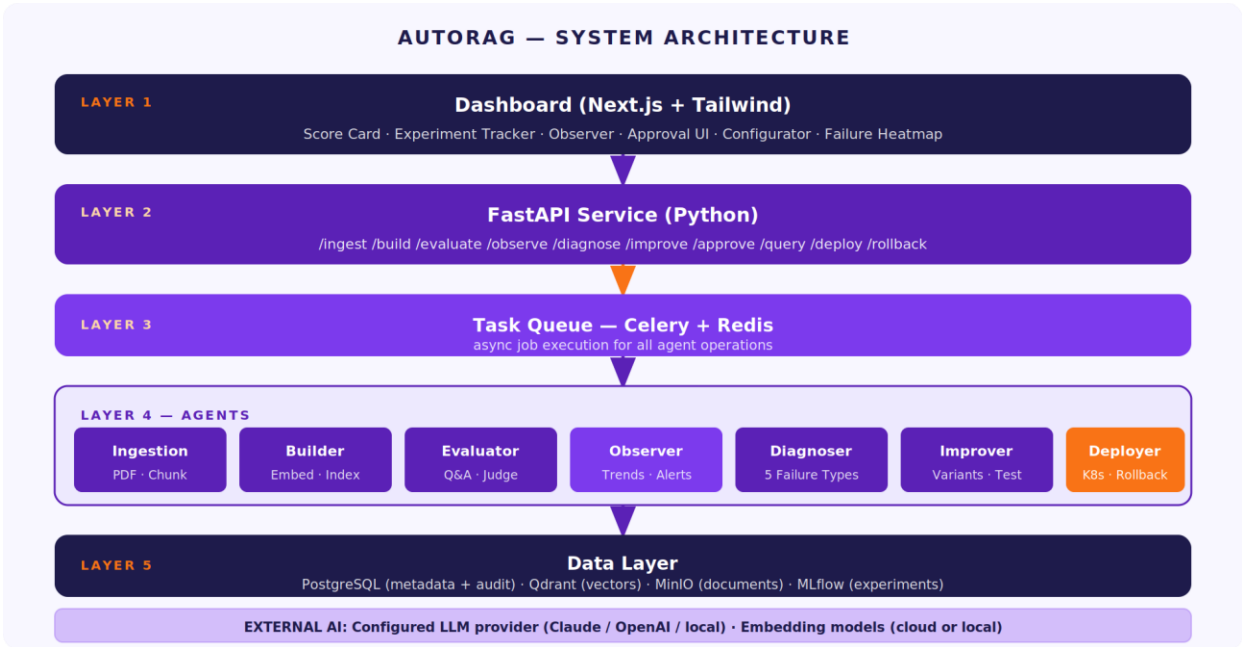


Figure 1: AutoRAG 5-Layer System Architecture — Dashboard to Data

Table 4: 5-Layer Architecture — Responsibility and Primary Components

Layer	Job	Primary Components	Talks To
Layer 1 — Dashboard	What users see. The face of the product.	Next.js 14, Tailwind CSS, shadcn/ui, Recharts	Layer 2 only (via REST API)
Layer 2 — API Service	Receives requests. Validates them. Dispatches work.	FastAPI, Pydantic, JWT/API-Key auth, OpenAPI	Layer 1 (responses) + Layer 3 (task dispatch)
Layer 3 — Task Queue	Runs long jobs asynchronously so API stays fast.	Celery, Redis broker, Flower monitoring	Layer 2 (receives tasks) + Layer 4 (runs agents)
Layer 4 — Agents	The intelligence layer. 7 specialised agents, each owning one stage.	7 Python agent modules — stateless, independently scalable	Layer 3 (triggered by) + Layer 5 (read/write state)
Layer 5 — Data	Where everything is stored. Permanent record of all decisions.	PostgreSQL, Qdrant, MinIO, MLflow, Redis (session)	Layer 4 only (agents read and write)

Why layers? Imagine the Evaluator Agent suddenly needs more compute because you uploaded 10 large documents at once. With layers, you simply add more Celery workers (Layer 3) — you do not touch the API or the database. Each layer scales independently.

04 Core Component Architecture

4.1 Component Responsibilities

Table 5: Core Components — Responsibility, Technology, and Scaling Model

Component	Technology	Responsibility	Scaling Model
API Service	FastAPI (Python)	14 REST endpoints; auth; request validation; async job dispatch	Horizontal — 3–10 replicas in production (HPA on CPU)
Task Queue	Celery + Redis	Async execution of all agent tasks; retry logic; job status tracking	Horizontal — 5–20 worker replicas (HPA on queue depth)
Ingestion Agent	Python + pypdf + Presidio	Load, clean, classify, chunk, PII-scan documents	Celery concurrency 4 per worker
Builder Agent	Python + OpenAI/MiniLM	Embed chunks; create Qdrant collection; store run record	Celery concurrency 2 (embedding API rate limit)
Evaluator Agent	Python + LLM judge	Generate 50 Q&A; score each; compute Unified Score	Celery concurrency 2; parallelised judge calls
Observer Agent	Python + PostgreSQL	Aggregate score trends; detect degradation; fire alerts	Runs as Celery beat scheduled task + on-demand
Diagnoser Agent	Python + LLM	Classify failures into 5 types; generate recommendations	Celery concurrency 4 (fast, low token usage)
Improver Agent	Python + Celery chain	Spawn 3–5 variant pipelines; compare scores; apply approval gate	Sequential — 1 per worker (expensive multi-step task)
Deployer Agent	Python + K8s API	Deploy best pipeline; run health check; trigger rollback if needed	Celery concurrency 1 (one deploy at a time)
Dashboard	Next.js 14 + Tailwind	20-screen UI; real-time score cards; approval queue; audit log	Horizontal — 3–8 replicas; CDN for static assets
PostgreSQL	PostgreSQL 15	All structured state — 9 core tables; append-only audit_log	Primary + 1 read replica in production; Patroni in P2
Qdrant	Qdrant (HNSW)	Vector store — 1 collection per pipeline run	Single node P1; multi-shard in P2 for >1M vectors
MinIO	MinIO (S3-compatible)	Raw documents + chunks — immutable object storage	Single node P1; distributed mode in P2
MLflow	MLflow OSS	Experiment artifact versioning; parameter and metric history	Single instance; shared across all pipeline runs
Redis	Redis 7	Celery broker; short-term session state (TTL 24h)	Single node P1; Sentinel cluster in production

4.2 API Endpoint Summary

All endpoints use base path `/api/v1/``, JSON bodies, TLS 1.3, and JWT Bearer or X-API-Key authentication.

Table 6: 14 REST API Endpoints — Method, Path, Agent Triggered

Method	Endpoint	Triggers	Returns
POST	/ingest	Ingestion Agent (async)	job_id for status polling
POST	/pipelines	Builder Agent (async)	pipeline_run_id
POST	/pipelines/{id}/evaluate	Evaluator Agent (async)	job_id → Unified Score when done
GET	/observe	Observer Agent (sync)	Score trend, failure distribution, cost data
POST	/diagnose	Diagnoser Agent (async)	Failure type + remediation recommendation
POST	/improve	Improver Agent (async)	Best variant + approval status
POST	/approve	Approval gate	Deploy triggered or rejection logged
POST	/configurator	Rules engine (sync, no LLM)	YAML config + scoring profile
GET	/experiments	PostgreSQL query	List of all pipeline runs with scores
POST	/experiments/compare	PostgreSQL query	Side-by-side score + param diff
POST	/query	RAG Runtime (sync)	Answer + source chunks + latency_ms
POST	/deploy/{run_id}	Deployer Agent (async)	Deployment record + health check status
POST	/deploy/rollback	Deployer Agent (sync)	Previous version reinstated
GET	/jobs/{job_id}	Job tracker	{status: pending running done failed, result}

05 7-Agent Autonomous Architecture

Each agent is a stateless Python module — it reads input from PostgreSQL or MinIO, does its job, writes output back, then exits. Agents do not communicate directly with each other. They communicate through shared state. This is like an assembly line — each station receives work, completes it, and passes the result to the next station via a shared conveyor belt (PostgreSQL).

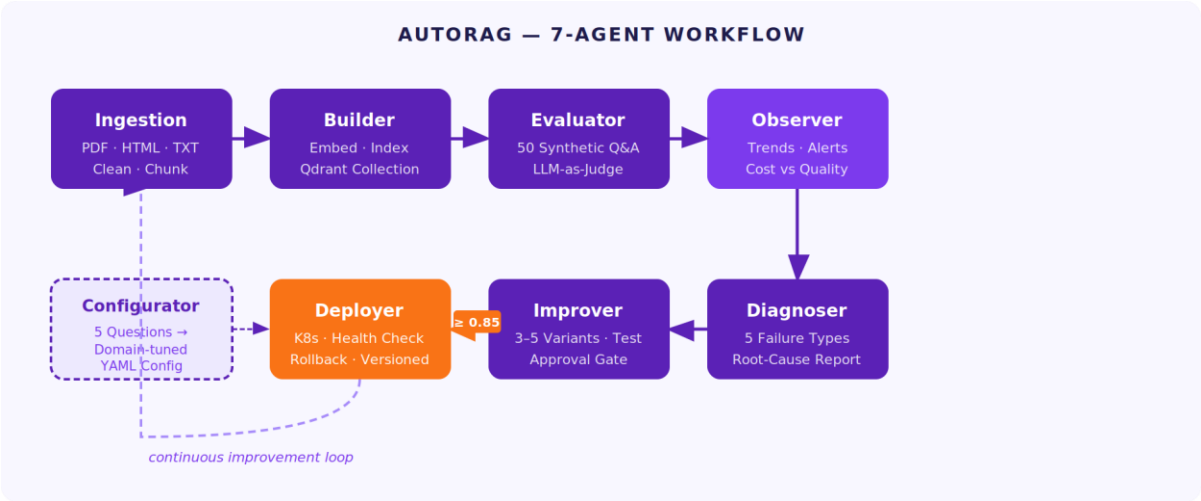


Figure 2: AutoRAG 7-Agent Closed-Loop Workflow with Configurator Entry Point

5.1 Agent Specification Table

Table 7: 7-Agent Architecture — Input, Output, Scaling, Key Integrations

Agent	Input	Output	Celery Concurrency	Key Integrations
Ingestion	Uploaded file path + config	Chunks in MinIO; metadata in PostgreSQL documents + chunks tables	4 per worker	pypdf, BeautifulSoup, Presidio PII, Camelot (tables), Tesseract (OCR)
Builder	pipeline_run_id + YAML config	Qdrant collection created; pipeline_runs row written	2 per worker (API rate limit)	OpenAI embeddings or MiniLM; Qdrant client; PostgreSQL
Evaluator	pipeline_run_id	evaluations rows (50); unified_score in pipeline_runs	2 per worker; judge calls parallelised	LLM judge (Claude/GPT-4o/Ollama); Qdrant search; PostgreSQL
Observer	pipeline_run_id + time window	Score trend data; alert flags; cost vs quality metrics	Beat scheduled + on-demand	PostgreSQL aggregation queries; alert threshold checks
Diagnoser	evaluation_id (failed questions)	diagnoses row; failure type + remediation recommendation	4 per worker	LLM classification call (low token); PostgreSQL; 200-case test set
Improver	diagnosis_id + current config	3–5 variant pipeline_run rows; approval_log entry	1 per worker (expensive)	Builder + Evaluator (spawned as sub-tasks); PostgreSQL; MLflow
Deployer	approved pipeline_run_id	deployments row; K8s config applied; health check result	1 per worker (serial)	K8s API or Docker Compose; health check POST /query x 10; rollback trigger

5.2 Plugin Architecture (Extension Points)

AutoRAG defines three abstract interfaces for community and enterprise extensions. Any third-party implementation that satisfies the interface contract can be registered and used without modifying core agent code.

Table 8: Three Plugin Interfaces — Purpose and Validation Requirement

Interface	Abstract Method	Built-in Implementations	Validation Gate
RetrievalStrategy	retrieve(query, top_k) → List[Chunk]	DenseRetrieval (Qdrant), SparseRetrieval (BM25), HybridRetrieval	Must pass golden dataset; register via autorag.plugins registry
EmbeddingModel	embed(texts) → List[Vector] + dimension property	OpenAIEmbedding (1536-dim), MiniLMEmbedding (384-dim)	Collection naming must include dim suffix to prevent mismatch
LLMJudge	score(question, answer, context) → JudgeResult	ClaudeJudge, GPT4oJudge, OllamaJudge	≥ 0.85 correlation with reference Claude judge on golden dataset

06 Data & Vector Architecture

Think of AutoRAG's data layer as four specialised filing systems, each optimised for a different kind of data. PostgreSQL is your structured records (spreadsheets). Qdrant is your similarity search engine (finds things that are semantically similar). MinIO is your file cabinet (stores the actual documents). MLflow is your lab notebook (tracks every experiment).

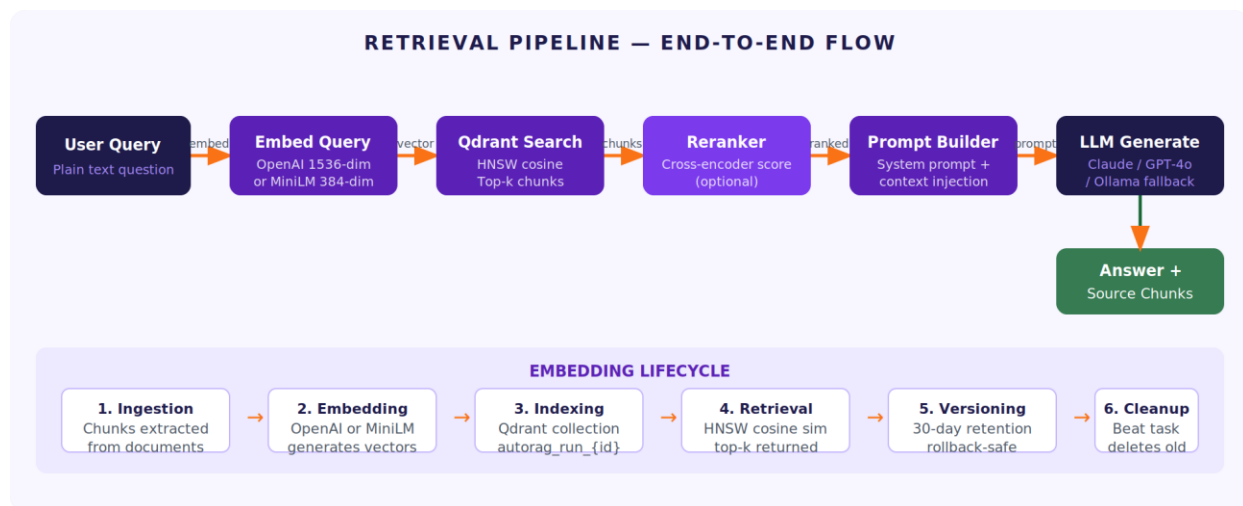


Figure 3: Retrieval Pipeline — User Query to Answer (with Embedding Lifecycle)

6.1 Storage Responsibility Matrix

Table 9: Four Stores — What Each Owns and Why

Store	Technology	What It Stores	Why This Store	Retention
Relational State	PostgreSQL 15	Pipeline configs (JSONB), eval results, audit log, 9 core tables	ACID guarantees; JSONB for config snapshots; row-level security in P2	Indefinite; daily backups; WAL archiving
Vector Embeddings	Qdrant (HNSW)	Chunk vectors — 1536-dim (OpenAI) or 384-dim (MiniLM); 1 collection per run	Purpose-built vector search; HNSW cosine; self-hostable; no cloud lock-in	30 days non-active; permanent for production collection
Object Storage	MinIO (S3-compatible)	Raw PDFs, HTML, chunk text, ML artifacts	Full data residency; S3-compatible API; works offline; immutable by convention	Per policy; 7 years enterprise; 7 days for exports
Experiment Tracking	MLflow OSS	Parameter history, metric runs, artifact versioning	Version-controlled experiment reproducibility; self-hosted; Python-native	Indefinite production; 90 days experimental
Session State / Queue	Redis 7	Celery task queue, active run session state	Sub-millisecond pub/sub; Celery native broker; TTL 24h auto-cleanup	TTL-based; 24h max for session; queue items consumed on completion

6.2 Qdrant Vector Store Design

Table 10: Qdrant Configuration Parameters

Parameter	Value	Why This Setting
Distance metric	Cosine similarity	Optimal for normalised text embeddings. Better than dot product for variable-length chunks.
Indexing algorithm	HNSW — m=16, ef_construct=200, ef_search=128	Fast approximate nearest-neighbour. m=16 balances graph density vs memory. ef_search=128 for high recall.
Collection naming	autorag_{pipeline_id}_{version}	One collection per pipeline version. Rollback = switch collection. No data migration needed.
Payload fields	chunk_id, source_doc_id, page_number, chunk_text, embedding_model	Enables filtered search by document, page, or model without fetching from PostgreSQL.
Retention policy	30 days (non-active); permanent (production)	Controlled by Celery beat cleanup task. Prevents unbounded storage growth during experiments.

6.3 Key Database Tables

Table 11: 9 PostgreSQL Core Tables — Purpose and Key Design Decisions

Table	Purpose	Key Design Decision
pipelines	YAML config snapshots; approval mode; scoring weights	JSONB for config — entire YAML stored as a single column. Never updated after creation.
pipeline_runs	One row per eval run — all 5 sub-scores + Unified Score	Immutable after creation. score fields are NOT NULL — evaluation always produces a score.
evaluations	Per-Q&A results — 50 rows per eval run	failure_type column set by Diagnoser; NULL until diagnosis runs.
experiments	Improvement variant history — links back to parent_run	variant_diff stored as JSONB. score_delta computed at insert time.
deployments	Active deployment record — one row per version	status field: active / superseded / rolled_back. Only one active per pipeline.
audit_log	Append-only event log — 11 event types	INSERT-ONLY enforced at database level. App user has no UPDATE or DELETE permission.
query_logs	Production /query traffic	Feeds Observer Agent trend analysis. Anonymised before LLM analysis.
agent_retries	Resilience log — every retry and circuit-breaker event	New in v2.3. Links to run_id for full traceability of provider failures.
doc_relationships	Cross-document citation graph	source→target with relationship_type (cites / supersedes / amends). CASCADE DELETE on pipeline deletion.

07 AI / LLM Architecture

AutoRAG never hard-codes a specific AI provider. The provider, model, and credentials are configured at deploy time via environment variables. This means an enterprise can run AutoRAG entirely with Claude, entirely on local Ollama models, or switch between providers without changing a single line of code.

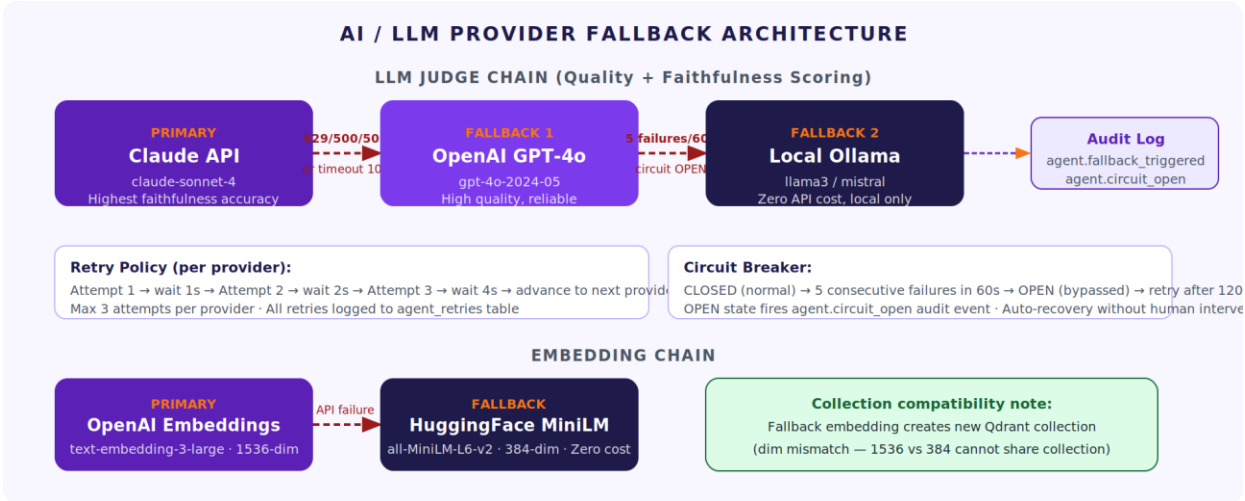


Figure 4: LLM Provider Fallback Chain — Primary, Fallback-1, Fallback-2 with Circuit Breaker

7.1 Provider Usage Matrix

Table 12: AI Provider Usage by Operation — Phase 1

Operation	Primary Provider	Fallback-1	Fallback-2	Est. Cost / 50-Q Eval
Q&A Generation	Claude Haiku	GPT-4o-mini	Local Ollama	~\$0.001
Quality Judge	Claude Sonnet	GPT-4o	Local Ollama	~\$0.09
Faithfulness Judge	Claude Sonnet	GPT-4o	Local Ollama	~\$0.09
Failure Classification	Claude Haiku	GPT-4o-mini	Local Ollama	~\$0.003
Cloud Embedding	OpenAI text-embedding-3-large (1536-dim)	—	HuggingFace MiniLM (384-dim)	~\$0.02/1K chunks
Local Embedding	MiniLM-L6-v2 (384-dim)	—	—	\$0.00 (local compute)
RAG Generation	/query endpoint — configurable at deploy time	—	—	~\$0.001–0.005 per query

7.2 Domain Scoring Profiles

Table 13: 4 Built-in Domain Scoring Profiles — Weight Overrides

Profile	Domain	Retrieval	Quality	Faithfulness	Latency	Cost	Use When
Faithfulness-heavy	Legal, Medical	0.20	0.30	0.45	−0.03	−0.02	Hallucination is a compliance risk
Speed-heavy	Customer Support	0.25	0.35	0.15	−0.20	−0.05	Response time is primary UX concern

Profile	Domain	Retrieval	Quality	Faithfulness	Latency	Cost	Use When
Completeness-heavy	Research, Internal KB	0.35	0.35	0.20	-0.05	-0.05	Comprehensive answers over speed
Balanced (default)	General purpose	0.25	0.35	0.25	-0.10	-0.05	No specific domain constraint

08 Workflow & Sequence Flows

These are the five key journeys through AutoRAG. Each is a sequence of steps — like a recipe. Read them left-to-right: what the user/system does, what happens next, where the result goes.

8.1 Document Ingestion Flow

Table 14: Ingestion Sequence — POST /ingest to Indexed Chunks

Step	Actor	Action	Output / State Change
1	User	POST /ingest with file upload	FastAPI receives file; validates MIME type
2	FastAPI	Dispatch ingest_document Celery task	Returns job_id immediately (async)
3	Ingestion Agent	Classify document pages (text/table/image/mixed)	Page classification written to chunk metadata
4	Ingestion Agent	Run Presidio PII scanner	PII detected → flagged in audit_log; processing paused for user decision
5	Ingestion Agent	Chunk document (fixed-size or semantic based on config)	Chunks stored in MinIO; metadata in PostgreSQL chunks table
6	Builder Agent	Embed chunks via OpenAI or MiniLM	Embedding vectors created (1536 or 384 dim)
7	Builder Agent	Upsert vectors into Qdrant collection	Collection autorag_{pipeline_id}_{version} created and populated
8	PostgreSQL	Write pipeline_runs row with config snapshot	Status: built; collection_name set; audit_log: pipeline.created

8.2 Query Lifecycle (POST /query)

Table 15: Query Sequence — Question to Answer with Source Chunks

Step	Latency Budget	Action	Notes
1. Embed query	< 100ms	OpenAI or MiniLM embeds user question	Same model as collection — dimension must match
2. Qdrant search	< 50ms	HNSW cosine similarity search; top-k chunks returned	Payload filter by doc_id if include_related_docs=true
3. Optional rerank	< 100ms	Cross-encoder reranks chunks if configured	Reduces context noise; increases faithfulness
4. Build prompt	< 5ms	System prompt + chunk context + user question assembled	Injection defence: chunks in delimited block
5. LLM generate	< 2,500ms	Configured provider generates answer	Fallback chain fires on timeout or 429/500/503
6. Log query	< 20ms (async)	query_logs row written; feeds Observer Agent	Non-blocking; does not add to user-perceived latency
7. Return	< 5ms	answer + source_chunks + latency_ms returned	related_docs appended if flag set
Total p95	< 3,000ms	End-to-end p95 SLO from PRD v2.3	Auto-rollback triggers if p95 > 5,000ms post-deploy

09 Evaluation & Improvement Loop

This is the heart of AutoRAG. Everything else exists to support this loop. The Evaluator measures quality, the Observer watches for degradation, the Diagnoser finds the root cause, and the Improver fixes it — automatically.

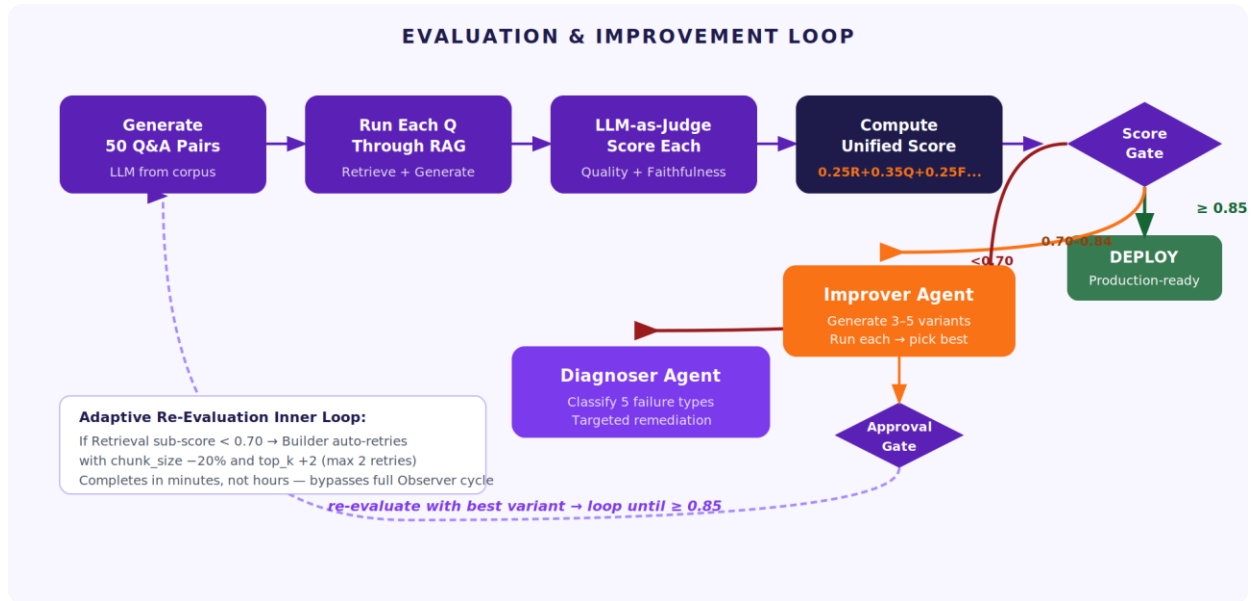


Figure 5: Evaluation & Improvement Loop — From Score to Approved Deployment

9.1 Unified Score Formula

$$\text{Score} = 0.25 \times \text{Retrieval} + 0.35 \times \text{Quality} + 0.25 \times \text{Faithfulness} - 0.10 \times \text{Latency} - 0.05 \times \text{Cost}$$

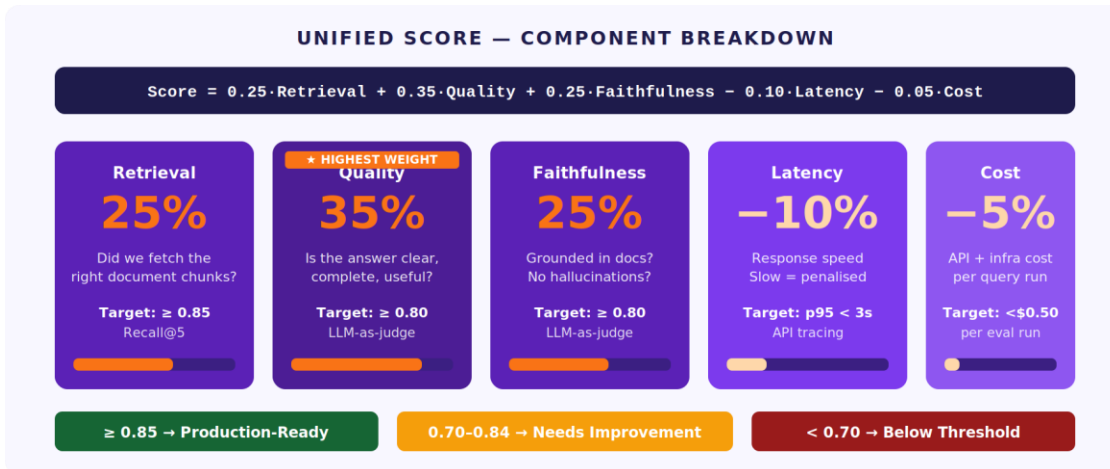


Figure 6: Unified Score Component Breakdown — Weights, Targets, Threshold Bands

Table 16: Score Thresholds and Automated System Actions

Score Range	Status	What Happens Automatically
≥ 0.85	Production-ready	Deploy via safe-deploy protocol. Flag as best version. Continue Observer monitoring.
0.70 – 0.84	Needs improvement	Trigger Improver Agent. Generate 3–5 variants. Test each. Submit best for approval.
< 0.70	Below threshold	Block deployment. Run Diagnoser. Surface failure type + remediation to dashboard.

9.2 Improvement Approval Framework

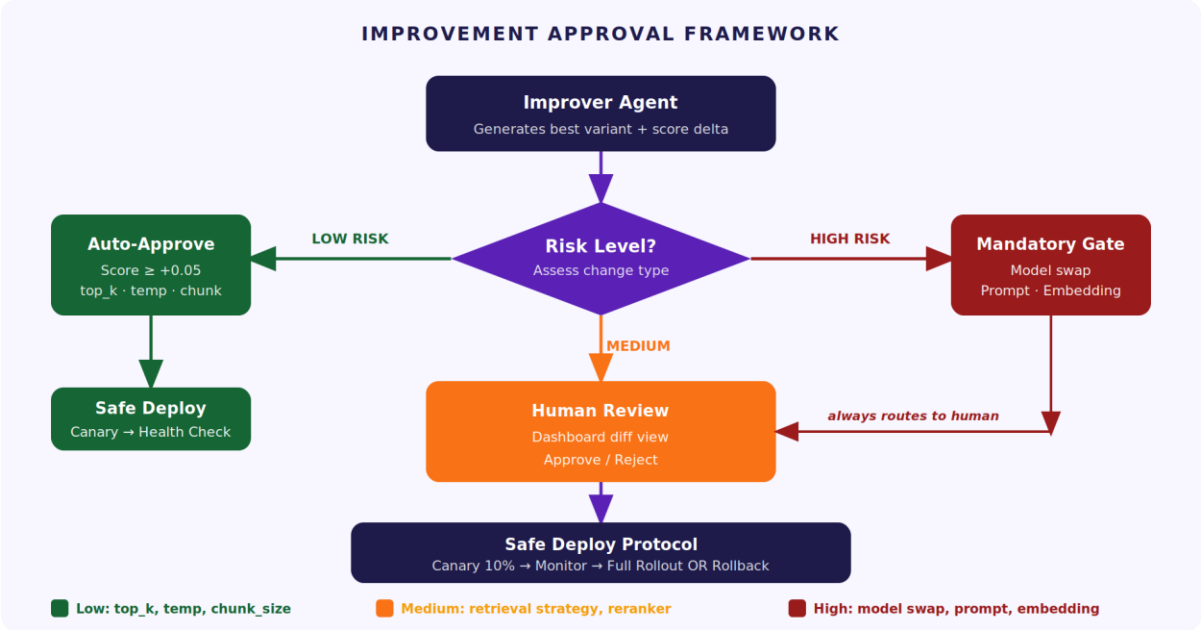


Figure 7: Improvement Approval Framework — Risk-Based Decision Tree

Table 17: Three Approval Modes — When Each Fires

Mode	Trigger Condition	What Happens	Can Be Bypassed?
Auto-Approve	Score improves ≥ 0.05 AND low-risk change (top_k, temp, chunk_size)	Safe deploy protocol executes automatically. audit_log: pipeline.approved.	Yes — enterprise setting can require human for all
Human-in-Loop	Enterprise governance mode OR medium-risk change (retrieval strategy swap)	Approval request surfaced in dashboard. Approver reviews diff + score delta.	No — must be resolved before deploy proceeds
Mandatory Gate	High-risk: model swap, system prompt change, embedding model change	Human approval always required. Cost ceiling check runs first. Data-residency rule enforced.	No — hard-coded guardrail in codebase

10 Security & Governance Architecture

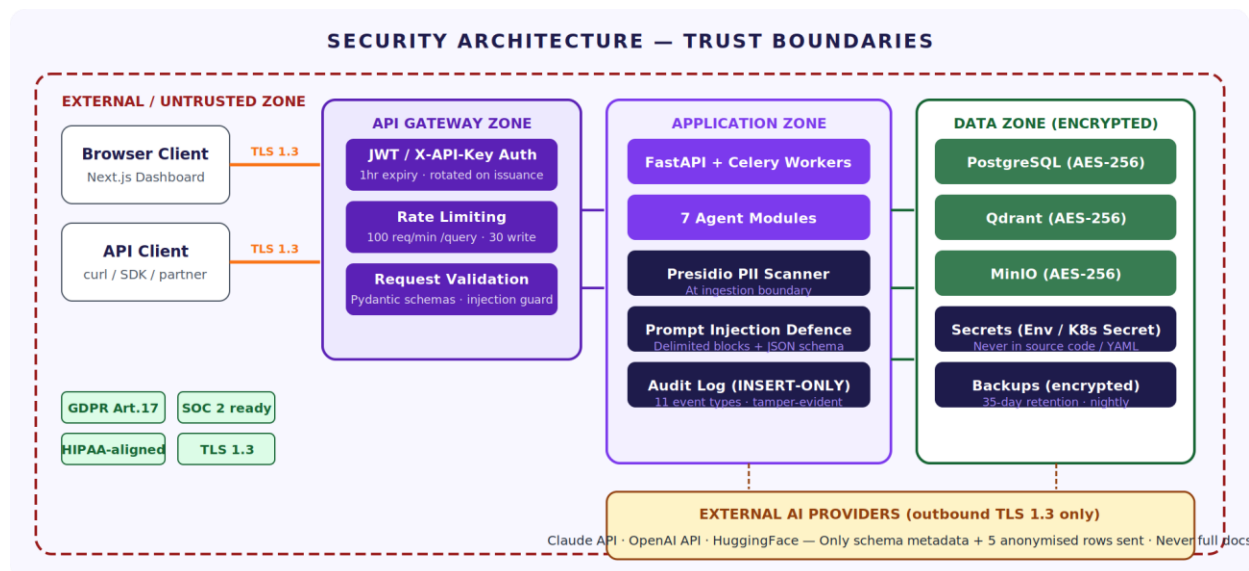


Figure 8: Security Architecture — Trust Boundaries and Data Protection Zones

10.1 Authentication & Authorisation

Table 18: Auth Controls by Component

Control	Mechanism	Detail
API Authentication	JWT Bearer (preferred) or X-API-Key	JWT: 1-hour expiry, 7-day refresh, RS256 in production; rotated on every issuance
Rate Limiting	Redis sliding window	100 req/min per key for /query; 30/min for write ops; 200-req burst for first 30s
PII Detection	Presidio at ingestion boundary	Detected PII → flagged in audit_log; processing halted; never enters Qdrant vector store
Prompt Injection	Delimited context blocks + JSON schema validation	System prompt isolated from corpus content; all LLM outputs validated before storage
Secrets	Env vars (dev) / K8s Secrets (prod)	Never in source code, YAML, or logs; masked in all log outputs
Encryption at rest	AES-256 on all PVC volumes	PostgreSQL, Qdrant, MinIO; customer-managed keys in P2
Encryption in transit	TLS 1.3 mandatory	All API calls, internal service-to-service in production, LLM provider calls
Audit log	INSERT-ONLY PostgreSQL table	11 event types; app user has no UPDATE or DELETE permission; tamper-evident

10.2 Governance Guardrails

- ▶ **Faithfulness < 0.5 blocks deployment** — hard-coded; cannot be disabled by config
- ▶ **Model upgrades are always Mandatory Gate** — Improver cannot auto-deploy a model switch
- ▶ **Cost ceiling check before model recommendation** — if projected cost > ceiling, recommendation blocked
- ▶ **Local inference hard-block** — if pipeline config has `inference\_mode: local`, cloud model never recommended
- ▶ **PII halts ingestion** — detected PII pauses processing until human decision logged in audit_log

11 Deployment & Scalability Architecture

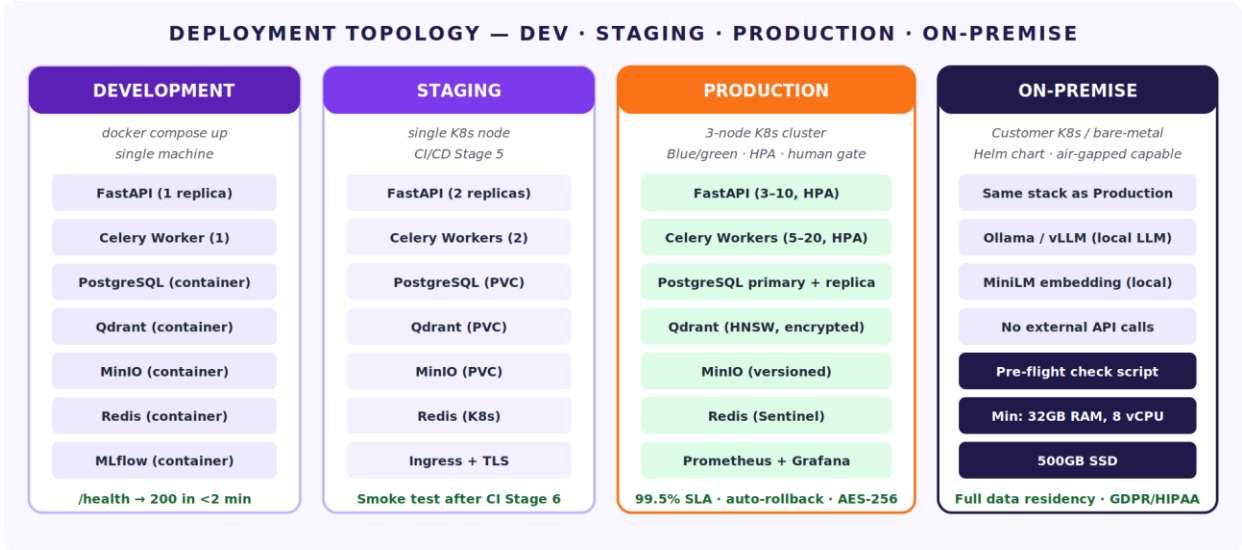


Figure 9: Deployment Topology — Development, Staging, Production, On-Premise

11.1 Environment Specifications

Table 19: Deployment Environments — Infrastructure and Purpose

Environment	Infrastructure	Min Spec	How to Start	Use Case
Development	Docker Compose — 7 containers on one machine	8 GB RAM, 4 vCPU, SSD	docker compose up	Solo dev; fast iteration; zero config
Staging	Single Kubernetes node	16 GB RAM, 4 vCPU	CI/CD Stage 5 (automated)	Integration testing; pilot demos; pre-release
Production	3-node K8s cluster; HPA; blue/green	3 × 32 GB RAM, 8 vCPU per node	CI/CD Stage 8 (post human approval)	Enterprise; HA; 99.5% SLO
On-Premise	Customer K8s / bare-metal; Helm chart; Ollama/vLLM for local LLM	32 GB RAM, 8 vCPU, 500 GB SSD (min)	Helm install + pre-flight check	Air-gapped; full data residency; GDPR/HIPAA

11.2 CI/CD Gate Chain

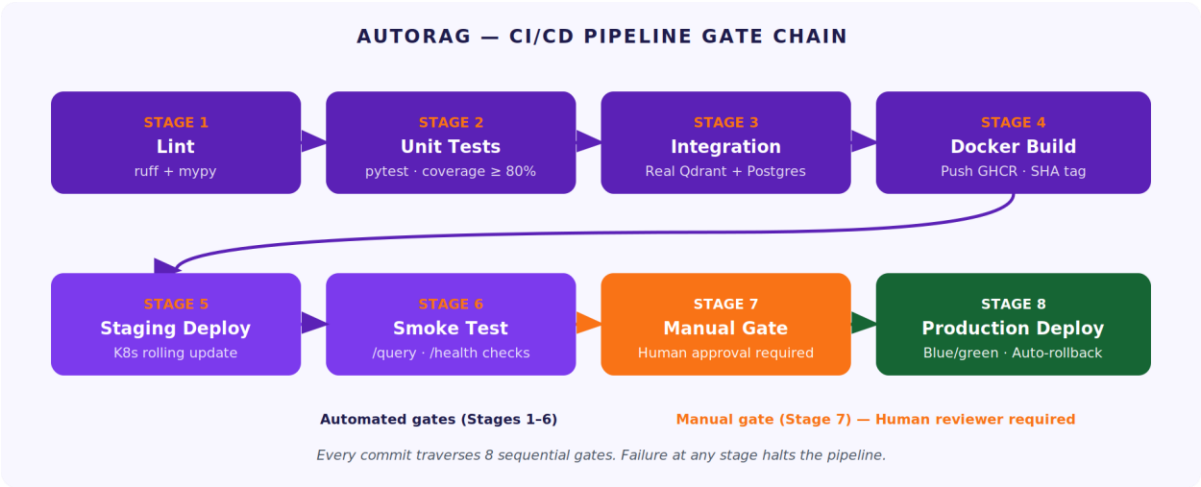


Figure 10: 8-Stage CI/CD Gate Chain — Automated Gates 1–6, Human Approval Gate 7

Table 20: CI/CD Stages — Tool, Action, Pass Criteria

Stage	Tool	Action	Pass Criteria
1 — Lint	ruff + mypy	Static analysis; type checking	Zero errors; no type violations
2 — Unit Tests	pytest	Module-level tests with coverage	Coverage ≥ 80%; all tests pass
3 — Integration	pytest + Docker Compose	Real Qdrant + PostgreSQL + Redis	All integration suites pass
4 — Docker Build	Docker + GHCR	Build API + worker + dashboard; SHA tag	Images build; size within threshold
5 — Staging Deploy	kubectl + Helm	Rolling deploy to staging	All pods Running; /health → healthy
6 — Smoke Test	pytest remote	POST /ingest → /build → /evaluate → /query	Unified Score ≥ 0.70; answer returned
7 — Manual Gate	GitHub Environments	Human reviewer approves prod deploy	Named reviewer explicit approval
8 — Prod Deploy	kubectl blue/green	Green deploy; traffic switch; blue 10-min standby	All pods healthy; p95 < 3s post-deploy

12 Observability & LLMOps

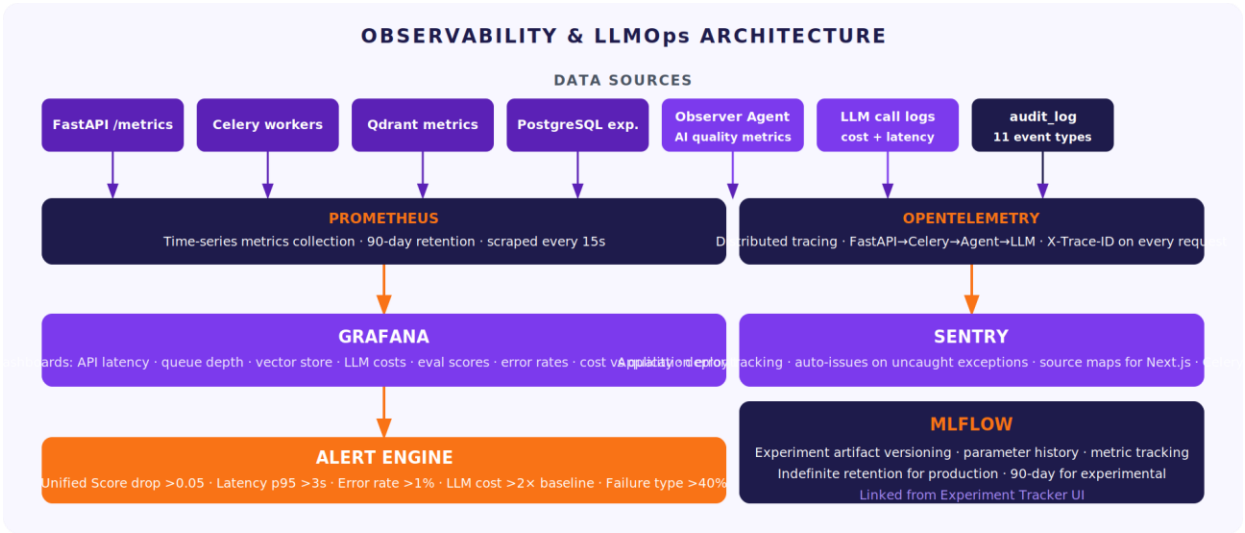


Figure 11: Observability Stack — Sources, Collection, Dashboards, Alerting, MLflow

12.1 AI-Specific Metrics Tracked by Observer Agent

Table 21: Observer Agent Alert Matrix

Metric	Source	Alert Threshold	Automatic Action
Unified Score trend (7-day)	Evaluator Agent	Drop > 0.05 vs baseline	Trigger Diagnoser → Improver chain
Query success rate	query_logs	< 0.95 over 1 hour	Alert + surface in Observer dashboard
Failure type concentration	Diagnoser Agent	Any type > 40% of failures	Surface dominant failure type prominently
LLM cost-per-query	Evaluator + query_logs	> 2× baseline for 4 hours	Alert; check if model upgrade was made
Latency p95 (rolling 15 min)	OpenTelemetry tracing	> 3 seconds	Alert; check Qdrant health and LLM provider
Faithfulness (production sample)	Periodic re-evaluation	< 0.70	Trigger full evaluation cycle

12.2 LLMops Continuous Loop

AutoRAG does not fine-tune models. Its LLMops loop continuously improves pipeline configuration, not model weights. The loop has two paths — a slow path through the Observer (hours) and a fast inner loop through the Evaluator (minutes).

Table 22: LLMops Two-Speed Improvement Architecture

Loop	Trigger	Path	Typical Duration
Outer Loop (standard)	Observer detects score drop > 0.05	Observer → Diagnoser → Improver → Approval → Deploy	4–6 hours end-to-end
Inner Loop (adaptive)	Retrieval sub-score < 0.70 on any eval	Evaluator → Builder (adjusted params: chunk_size -20%, top_k +2) → Re-evaluate	Minutes; max 2 retries
Scheduled Re-evaluation	Weekly Celery beat task	Evaluator runs full 50-Q cycle against production pipeline	5 minutes; feeds Observer trend data

13 Resilience & Failover Design

AutoRAG is designed to keep working even when external services fail. Think of it like a car with a spare tyre and roadside assistance — the primary solution is preferred, but there are always fallbacks. Every failure event is logged so you can see exactly what happened and why.

13.1 Provider Resilience Envelope

Table 23: Resilience Controls — Retry, Circuit Breaker, Fallback

Control	Mechanism	Configuration	Logged To
Retry policy	Exponential backoff per provider	Attempts 1→2→3; waits 1s→2s→4s; max 3 per provider before advancing	agent_retries table: all attempts
Circuit breaker	Provider marked OPEN after 5 failures in 60s	Re-tests after 120s; auto-recovery; HALF-OPEN state for test	audit_log: agent.circuit_open
LLM failover	Claude → GPT-4o → Local Ollama	Triggers on HTTP 429, 500, 503 or timeout > 10s	audit_log: agent.fallback_triggered
Embedding failover	OpenAI → HuggingFace MiniLM	Automatic on API failure; new Qdrant collection created (different dim)	audit_log: agent.fallback_triggered
Retrieval fallback	Dense → Hybrid if Retrieval score < 0.70	Max 2 auto-retries; surfaced as Retrieval Miss if both fail	agent_retries; then Diagnoser classifies
Pipeline rollback	Previous Qdrant collection + config reinstated	Auto: score drop >0.10, p95 >5s, error rate >1%; Manual: POST /deploy/rollback	audit_log: pipeline.rolled_back

13.2 Rollback Strategy

- ▶ **Every pipeline version is preserved** — Qdrant collection per version; PostgreSQL snapshot per run; 30-day retention
- ▶ **Rollback is a config switch** — route traffic to previous Qdrant collection; no data migration needed
- ▶ **Three auto-rollback triggers** — Unified Score drop > 0.10 post-deploy; p95 latency > 5s; API error rate > 1%
- ▶ **Recovery time < 2 minutes** — rollback is a single API call (POST /deploy/rollback) or one dashboard click
- ▶ **Blue/green in production** — new version deployed to green environment; traffic switched after health check; blue retained 10 min

14 Technology Stack

Table 24: Complete Technology Stack — Component, Version, Purpose, Layer

Component	Technology + Version	Purpose	Layer
API Framework	FastAPI (Python 3.11)	REST API; async; Pydantic validation; auto OpenAPI docs	API
Task Queue	Celery 5 + Redis 7	Async agent execution; retry logic; job status	Task Queue
Relational DB	PostgreSQL 15	All structured state; 9 core tables; audit log	Data
Vector Store	Qdrant (latest stable)	HNSW vector search; 1 collection per pipeline run	Data
Object Storage	MinIO (RELEASE.2024)	S3-compatible; raw docs + chunks; immutable	Data
Experiment Tracking	MLflow 2.x	Artifact versioning; parameter + metric history	Data
Cache / Broker	Redis 7	Celery broker; session state TTL 24h	Task Queue / Data
Frontend	Next.js 14 + Tailwind CSS + shadcn/ui	20-screen dashboard; SSR; real-time score cards	Dashboard
Charts	Recharts	Score trend charts; failure heatmaps; cost graphs	Dashboard
Cloud Embedding	OpenAI text-embedding-3-large	1536-dim embeddings for maximum retrieval accuracy	Agents
Local Embedding	HuggingFace MiniLM-L6-v2	384-dim; zero cost; local fallback	Agents
LLM Primary	Claude Sonnet 4 (Anthropic)	Q&A judge; faithfulness judge; RAG generation	Agents
LLM Fallback-1	OpenAI GPT-4o	LLM fallback when Claude unavailable	Agents
LLM Fallback-2	Ollama (local)	Air-gapped / cost-zero LLM fallback	Agents
PII Detection	Microsoft Presidio	PII scan at ingestion boundary	Agents
Tracing	OpenTelemetry SDK	Distributed tracing across all layers	Observability
Metrics	Prometheus + Grafana	8 dashboards; 90-day retention; alerting	Observability
Error Tracking	Sentry	Auto-issues on exceptions; source maps for Next.js	Observability
CI/CD	GitHub Actions	8-stage gate chain; GHCR image push	Deployment
Container	Docker + Kubernetes	Dev: Compose; Staging: single node; Prod: 3-node cluster	Deployment
K8s Config	Helm charts	Templated K8s deployment; one-click installer in P2	Deployment
Type Checking	mypy (strict)	Python type safety enforced in CI	Engineering
Linting	ruff	Fast Python linter; replaces flake8 + black	Engineering

15 Architecture Decision Records

An Architecture Decision Record (ADR) explains why a technology was chosen. Each one below follows the same format: Decision → Rationale → Alternative Considered. Read these if you ever wonder 'why did we use X instead of Y?'

Table 25: 7 Architecture Decision Records

#	Decision	Rationale	Alternative Considered
ADR-01	FastAPI over Django REST	Async-native; Pydantic validation built-in; auto-generates OpenAPI docs; faster for high-concurrency agent dispatch	Django REST — more batteries included but sync-first; heavier for this use case
ADR-02	Qdrant over pgvector or Pinecone	Self-hostable (no cloud lock-in); dedicated HNSW indexing; payload filtering without joining PostgreSQL; collection-per-run enables version-safe rollback	pgvector — simpler ops but no dedicated index tuning; Pinecone — SaaS lock-in, no data residency
ADR-03	PostgreSQL over MongoDB	ACID guarantees for audit log; JSONB for flexible config snapshots without schema migration; RLS foundation for P2 multi-tenant	MongoDB — flexible but weaker ACID; audit log integrity not guaranteed
ADR-04	Celery + Redis over Temporal	Python-native; simpler ops; no separate Temporal server; sufficient for AutoRAG's linear agent DAG; Redis already needed for session state	Temporal — more powerful state machine but adds operational complexity; overkill for P1
ADR-05	MLflow over Weights & Biases	Fully self-hosted; no SaaS dependency; Python-native; sufficient for experiment tracking without LLM fine-tuning	Weights & Biases — better UI but requires external SaaS account; violates data-residency for enterprise
ADR-06	Next.js over React SPA	Server-side rendering for faster initial load; file-based routing matches 20-screen inventory; Tailwind CSS + shadcn/ui for fast component development	React SPA (Vite) — simpler but slower initial load; no SSR; more bundler configuration
ADR-07	7-Agent design over monolith	Each agent is independently scalable, testable, and replaceable; failure isolation — Diagnoser failure does not affect /query; Plugin interfaces enable community extensions	Single service — simpler initially but becomes a bottleneck as evaluation and improvement tasks grow

16 Future Architecture Roadmap

Table 26: Phase 2 and Phase 3 Architecture Extensions

Phase	Feature	Architectural Change	Target Date
P2	Multi-model support (Gemini, Mistral, Llama)	Add providers to LLM abstraction layer; update fallback chain; re-calibrate scoring benchmark	Sept 2026
P2	Kubernetes one-click installer	Package Helm chart; add pre-flight check script; on-premise spec (32GB RAM, 8 vCPU, 500GB SSD)	Sept 2026
P2	RBAC + SSO	Role-based access on FastAPI middleware; SAML 2.0 / OIDC in auth layer; user table in PostgreSQL	Sept 2026
P2	Prometheus + Grafana managed	Add Prometheus operator to K8s Helm chart; 8 pre-built Grafana dashboards shipped with installer	Sept 2026
P2	PostgreSQL read replica + Patroni HA	Primary/replica failover; connection pooling via PgBouncer in Helm chart	Sept 2026
P2	Qdrant multi-shard	Horizontal Qdrant scaling for corpora > 1M vectors; collection routing layer in Builder Agent	Sept 2026
P3	SaaS Cloud tier	Multi-tenant PostgreSQL (schema-per-tenant); tenant-scoped Qdrant namespaces; managed LLM cost budgeting	Dec 2026
P3	SOC 2 Type II audit	Formalise audit log export; pen-test; access review automation; audit evidence collection tooling	Dec 2026
P3	Adaptive scoring weights	Re-calibrate Unified Score weights after each pilot cohort using production query logs; weights versioned	Dec 2026
P3	SharePoint + Confluence connectors	Ingestion Agent plugin for enterprise document sources; OAuth 2.0 auth; incremental sync	Dec 2026

17 Conclusion

AutoRAG's architecture is built around one idea: **a RAG system that never stops improving itself**. The 7-agent closed loop — Ingest, Build, Evaluate, Observe, Diagnose, Improve, Deploy — is the engine. The Unified Score is the fuel gauge. The approval framework is the safety net.

Every architecture decision in this document — from the choice of Qdrant over pgvector, to the use of Celery over Temporal, to the plugin interface design — traces back to three principles: zero cloud lock-in, immutable audit trail, and safe by default. These principles make AutoRAG deployable anywhere, trustworthy for regulated industries, and extensible by the open-source community.

For a beginner reading this document: start with Section 3 (the 5-layer architecture), then Section 5 (the 7 agents), then Section 9 (the evaluation loop). Those three sections contain the core ideas. Everything else is implementation detail that you will need when you build each component.

The architecture is designed to be built in 45 days by 1–2 engineers. It is not over-engineered for Phase 1 — the goal is a working, measurable, improvable RAG system. The complexity in Sections 11–13 is Phase 2 and Phase 3 work. Ship Phase 1 first.

AutoRAG | Enterprise Architecture Design Document v1.0 | Derived from PRD v2.3 | May 2026

CONFIDENTIAL — For Engineering & Product Review | Euron AI Product Engineering 2.0

Prepared by Euron Architecture Team | May 2026 | architecture@euron.one